Модули, пакеты, сборка

Как устроены инструменты фронтенда



Frontend-разработчик

Что будем уметь

- Объяснять, как браузер и Node находят импортируемый модуль
- Читать package.json и lock-файл, понимать semver
- Объяснять, зачем нужна сборка и что она делает с кодом
- Отличать режим разработки от production-сборки
- Понимать, как работают линтер и форматтер

Демонстрация после лекции: переносим «Жёлтого мага» в проект Vite

Чего хочется от проекта

Задача	Инструмент
Подключать и обновлять чужой код	npm
Разработка с мгновенным обновлением	dev-сервер Vite
Быстрая загрузка у пользователя	сборка: vite build
Находить ошибки до запуска	ESLint
Одинаковое форматирование у всех	Prettier

Все эти инструменты — программы на JavaScript, запускаемые в Node

Как появились модули

Подход	Как	Проблема
<script> подряд	всё в глобальной области	конфликты имён, порядок файлов
Пространство имён	window.Game = { … }	зависимости вручную
CommonJS	require, module.exports	синхронный, придуман для Node
ES modules	import, export	стандарт для браузера и Node

ESM — стандарт, но CommonJS ещё встретится в пакетах

CommonJS и ES modules

```
// CommonJS
const confetti = require("canvas-confetti");
module.exports = { castSpell };

// ES modules
import confetti from "canvas-confetti";
export function castSpell() {}
```

require — обычная функция, выполняется во время работы.

import — синтаксис, известен до запуска.

Статические import позволяют построить граф модулей до запуска

Что можно написать после from

Спецификатор	Пример	Кто понимает
Относительный	"./rules.js"	браузер и Node
От корня сайта	"/src/rules.js"	браузер
Полный URL	"https://cdn…/confetti.mjs"	браузер
Имя пакета (bare)	"canvas-confetti"	Node и сборщики
Встроенный модуль	"node:fs"	только Node

Браузер понимает только URL. Имя пакета — соглашение Node

Браузер и имя пакета

```
<script type="module">
  import confetti from "canvas-confetti";
</script>
```

```
Uncaught TypeError: Failed to resolve module specifier
"canvas-confetti". Relative references must start with
either "/", "./", or "../".
```

Браузер не знает, где лежит пакет.

node_modules для него — просто папка.

Имя пакета нужно превратить в URL

Как Node ищет пакет

```
import "canvas-confetti" из /game/src/main.js

/game/src/node_modules/canvas-confetti  — нет
/game/node_modules/canvas-confetti      — есть
  ↓ package.json пакета
  "exports"?  — нет
  "main": "src/confetti.js"
  ↓
/game/node_modules/canvas-confetti/src/confetti.js
```

Поднимаемся по папкам вверх, ищем node_modules.

В package.json пакета узнаём, какой файл главный.

Алгоритм поиска — соглашение Node, которое повторяют сборщики

Поле exports

```
{
  "name": "spellbook",
  "type": "module",
  "exports": {
    ".": {
      "import": "./dist/index.js",
      "require": "./dist/index.cjs"
    },
    "./fire": "./dist/fire.js"
  }
}
```

Пакет сам решает, какие файлы видны снаружи.

Для import и require — разные файлы.

exports — публичный интерфейс пакета

Import maps: имена без инструментов

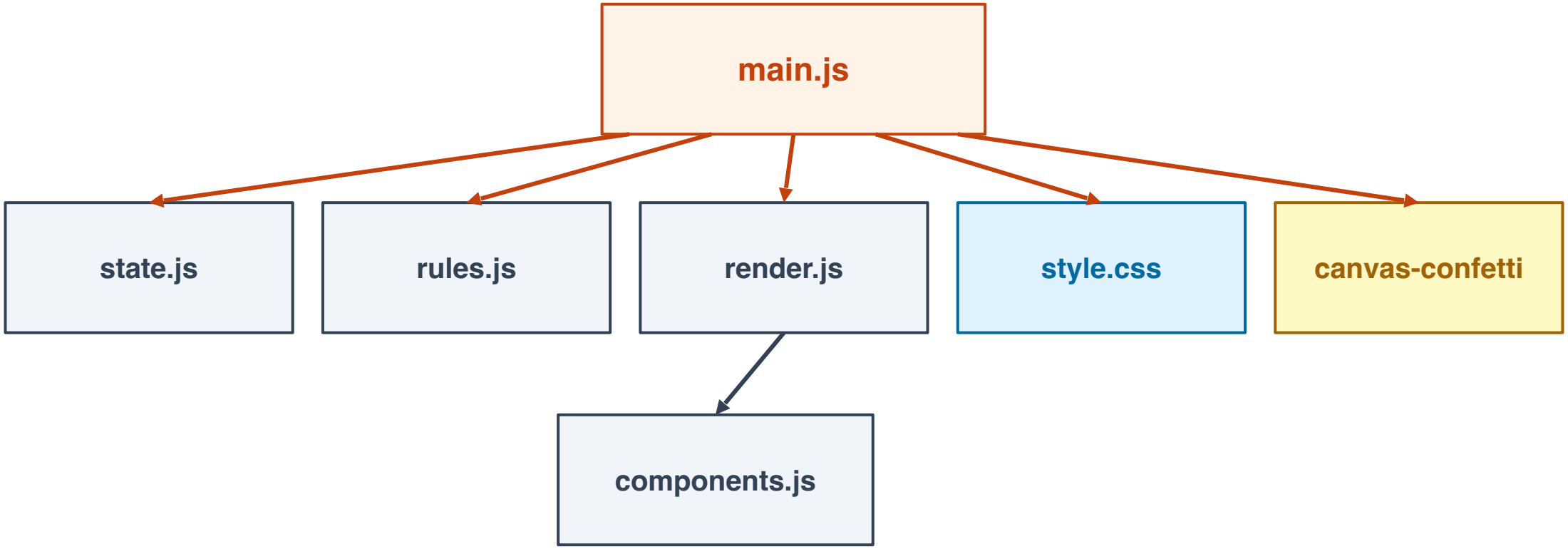
```
<script type="importmap">
 {
   "imports": {
     "canvas-confetti": "/vendor/confetti.module.mjs"
   }
 }
</script>

<script type="module">
  import confetti from "canvas-confetti";
</script>
```

Браузер понимает имена пакетов, если явно сказать, где файл.

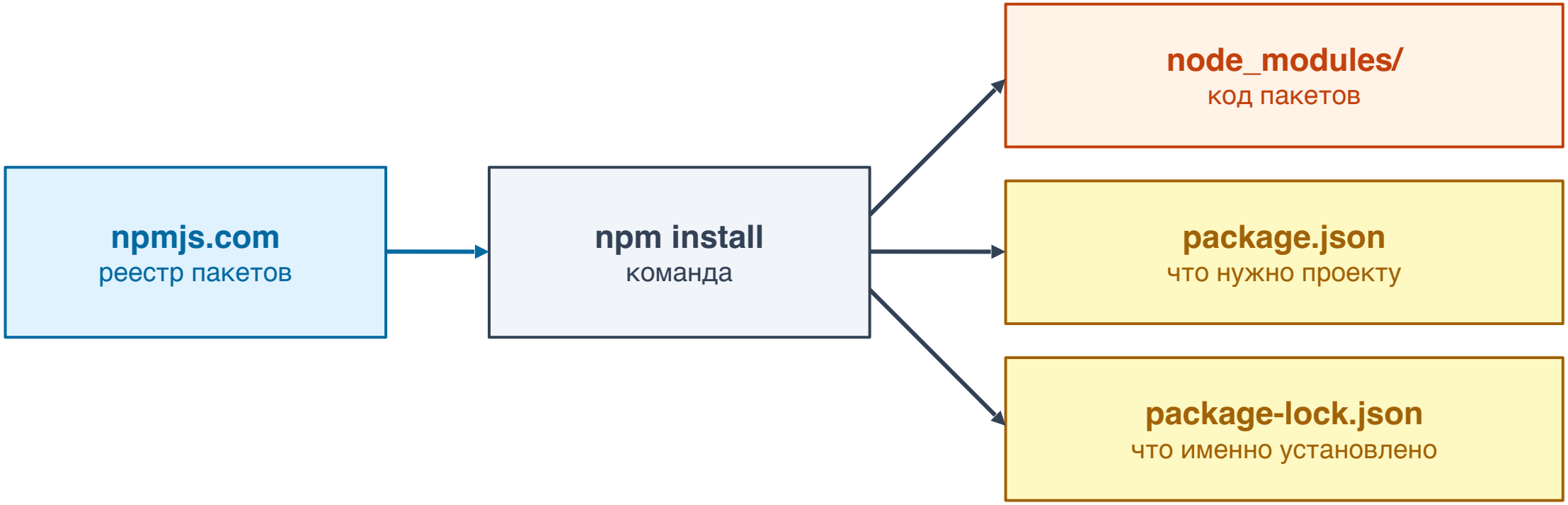
Стандарт без сборки, но пути и обновления — вручную

Граф модулей



Приложение — граф модулей с одной точкой входа

npm: реестр и программа



npm скачивает пакеты и записывает, что скачано

package.json

```
{
  "name": "yellow-mage",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": { "canvas-confetti": "^1.9.4" },
  "devDependencies": { "vite": "^8.3.4" }
}
```

Паспорт проекта: имя, скрипты, зависимости.

private — защита от случайной публикации.

Проект для npm — папка с package.json

Дерево зависимостей

```
$ npm ls --depth=0
yellow-mage@0.0.0
├── @eslint/js@10.0.1
├── canvas-confetti@1.9.4
├── eslint@10.13.0
├── globals@17.13.0
├── prettier@3.9.10
└── vite@8.3.4

6 прямых зависимостей → 98 пакетов в node_modules
```

Пакеты зависят от других пакетов.

Мы ставим 6, npm приносит 98.

Каждая зависимость — чужой код в вашем проекте

Semver: что значит ^1.9.4

Запись	Какие версии подходят
1.9.4	ровно 1.9.4
~1.9.4	1.9.x, начиная с 1.9.4
^1.9.4	1.x.x, начиная с 1.9.4
MAJOR.MINOR.PATCH	ломающие изменения . новое . исправления

По умолчанию npm пишет ^ — разрешены совместимые обновления

package-lock.json

```
"node_modules/canvas-confetti": {
  "version": "1.9.4",
  "resolved": "https://registry.npmjs.org/canvas-confetti/
              -/canvas-confetti-1.9.4.tgz",
  "integrity": "sha512-yxQbJkAVrFXWNbTUjPqjF7G+g6…",
  "license": "ISC"
}
```

package.json — какие версии подходят, lock-файл — какие именно установлены

npm install и npm ci

	npm install	npm ci
Источник версий	package.json и lock	только lock-файл
Меняет lock-файл	может	нет, ошибка при расхождении
node_modules	дополняет	удаляет и ставит заново
Когда	при разработке	CI, проверка, деплой

npm ci — воспроизводимая установка ровно тех же версий

Что хранить в git

Файл или папка	В git?	Почему
package.json	да	описание проекта
package-lock.json	да	одинаковые версии у всех
node_modules/	нет	восстанавливается из lock-файла
dist/	нет	собирается командой build

node_modules и dist — в .gitignore

npm scripts

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint ."
}

$ npm run build
$ npx vite --version   # разовый запуск
```

npm добавляет node_modules/.bin в PATH на время скрипта.

Глобальная установка не нужна.

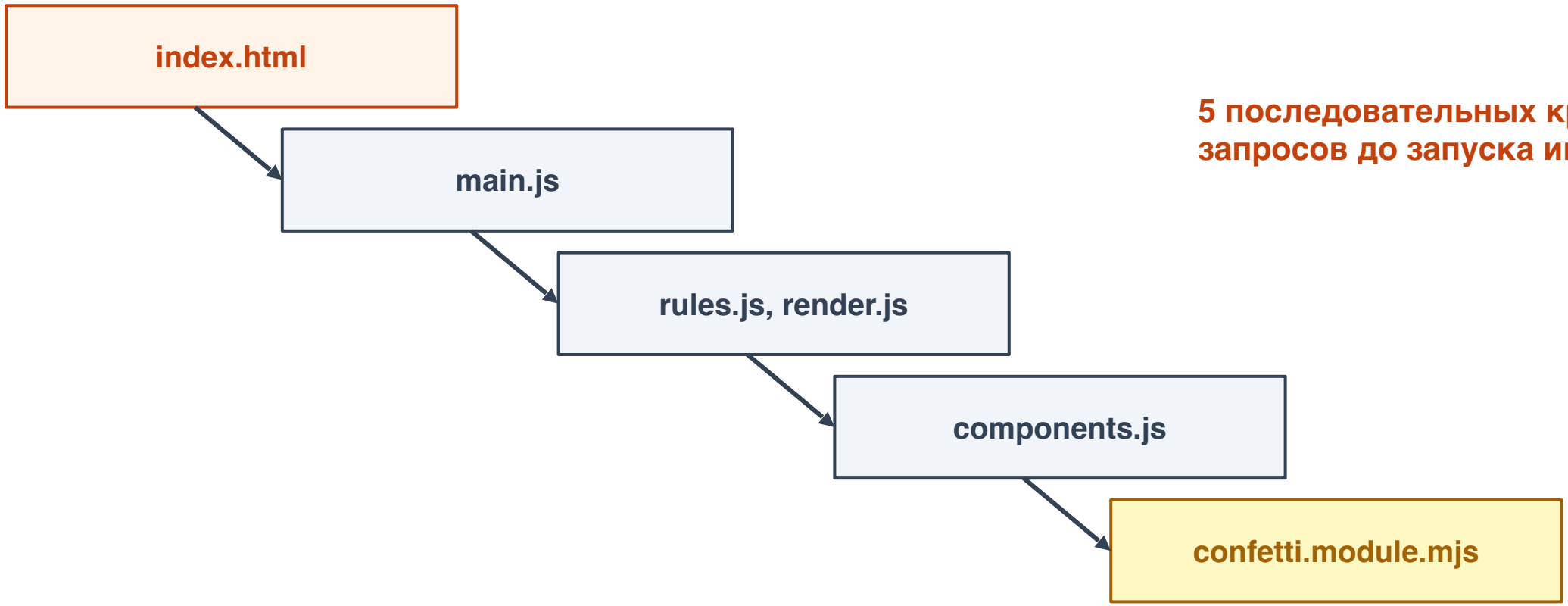
У всех одна версия инструмента — из lock-файла

Риски чужого кода

Угроза	Как выглядит	Что делать
Install-скрипты	postinstall выполняется при установке	--ignore-scripts, проверять пакет
Тайпсквоттинг	canvas-confeti вместо canvas-confetti	копировать имя из документации
Взлом автора	вредная версия популярного пакета	lock-файл, npm audit
Уязвимость	ошибка в старой версии	npm audit, обновления

npm install запускает чужой код на вашем компьютере

Водопад запросов



5 последовательных кругов запросов до запуска игры

Глубина графа = число последовательных запросов

Что делает сборщик

Шаг	Что происходит
Бандлинг	граф модулей → несколько файлов
Tree-shaking	неиспользуемые export выбрасываются
Минификация	короткие имена, без пробелов и комментариев
Хеши в именах	новое содержимое → новое имя файла
Транспиляция	синтаксис под целевые браузеры
Ресурсы	CSS и картинки — тоже часть графа

Сборка оптимизирует код для загрузки пользователем

Tree-shaking

```
// spells.js
export function spark(power) {
  return power * 2;
}
export function meteor() {
  return "METEOR_UNUSED";
}

// main.js
import { spark } from "./spells.js";
console.log(spark(3));
```

meteor нигде не импортируется.

В собранном файле строки METEOR_UNUSED нет.

Tree-shaking возможен благодаря статическим import

Минификация

```
// spells.js
export function spark(power) {
  const damage = power * 2;
  return damage;
}
// main.js
import { spark } from "./spells.js";
const mage = { name: "Жёлтый маг", power: 3 };
console.log(`${mage.name}: ${spark(mage.power)}`);

// после vite build: один файл
function e(e){return e*2}var t={name:`Жёлтый маг`,
power:3};console.log(`${t.name}: ${e(t.power)}`);
```

Имена сокращены, лишняя переменная damage исчезла.

Меньше байтов — быстрее загрузка

Хеши в именах и кеширование

```
dist/index.html
dist/assets/index-CWnWgquD.js
dist/assets/index-CxBcSoam.css

index.html                Cache-Control: no-cache
assets/index-CWnWgquD.js  Cache-Control: max-age=31536000,
                                         immutable
```

Файл с хешем можно кешировать навсегда.

Изменилось содержимое — изменилось имя.

index.html всегда проверяем, ассеты с хешем — кешируем навсегда

Source maps

```
$ vite build --sourcemap

dist/assets/index-Cj5hdUnw.js
dist/assets/index-Cj5hdUnw.js.map

// последняя строка собранного файла
//# sourceMappingURL=index-Cj5hdUnw.js.map
```

Карта связывает позиции в сжатом коде с исходниками.

DevTools показывает исходный код и строку ошибки.

Отлаживаем минифицированный код как исходный

Транспиляция и целевые браузеры

```
// исходник
const name = mage.spell?.name ?? "Без заклинания";

// для старого браузера без ?. и ?? — примерно так
var _a, _b;
var name = (_b = (_a = mage.spell) == null ? void 0
  : _a.name) != null ? _b : "Без заклинания";
```

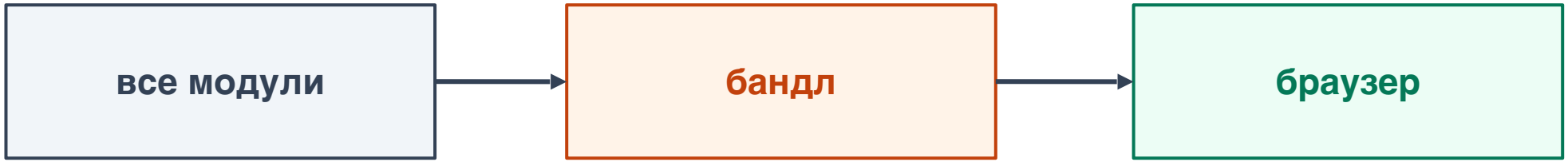
Новый синтаксис переписывается для старых браузеров.

Новые API (функции) так не появятся — для них нужны полифиллы.

Vite по умолчанию: Chrome 111, Firefox 114, Safari 16.4

Два подхода к режиму разработки

Бандлер в разработке



Vite в разработке



Vite быстро стартует: работа делается только для запрошенных модулей

Что dev-сервер делает с импортом

```
// src/main.js — как написано
import confetti from "canvas-confetti";
import mageUrl from "./assets/mage.png";

// что получил браузер
import confetti from
  "/node_modules/.vite/deps/canvas-confetti.js?v=9d10937a";
import mageUrl from "/src/assets/mage.png?import";
```

Vite превращает имена пакетов в URL — то, чего не умеет браузер

Предварительная сборка зависимостей

```
node_modules/.vite/deps/
  canvas-confetti.js
  _metadata.json

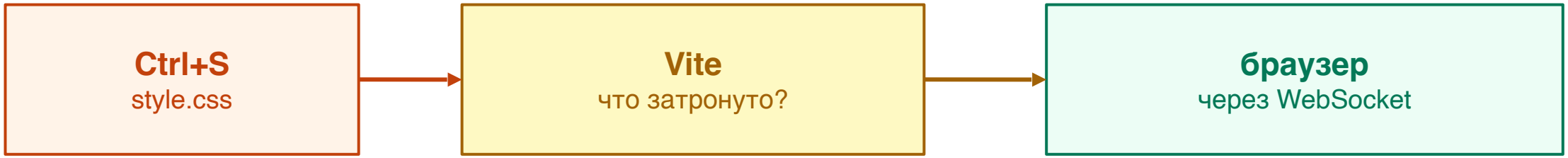
// lodash-es — более 600 внутренних модулей
import { debounce } from "lodash-es";
// без пресборки: сотни запросов при старте
```

Зависимости собираются один раз при первом запуске.

CommonJS превращается в ESM.

Свой код — по модулям, чужой — заранее собранным

Hot Module Replacement




```
if (import.meta.hot) {
  import.meta.hot.accept((newModule) => {
    // заменить код, сохранив состояние игры
  });
}
```

Модуль заменяется на лету, состояние страницы сохраняется

Разработка и production

	npm run dev	npm run build
Модули	по одному, как в src	собраны в несколько файлов
Код	как написан	минифицирован
Имена файлов	исходные	с хешем
Где работает	dev-сервер Vite	любой статический хостинг
Оптимизирован для	скорости обновления	скорости загрузки

Перед сдачей проверяем сборку: npm run build, затем npm run preview

Результат сборки

```
$ npm run build

✓ 12 modules transformed.
dist/index.html                  1.12 kB  |  gzip: 0.60 kB
dist/assets/mage-gkCPYza9.png   183.82 kB
dist/assets/index-CxBcSoam.css    1.61 kB  |  gzip: 0.75 kB
dist/assets/index-CWnWgquD.js    16.07 kB  |  gzip: 6.55 kB
✓ built in 106ms
```

dist — обычные статические файлы, Node для запуска не нужен

Код как дерево: AST

```
const mana = 20;

VariableDeclaration  kind: "const"
 └── VariableDeclarator
    ├── id:   Identifier  name: "mana"
    └── init: Literal     value: 20
```

Инструменты работают не с текстом, а с синтаксическим деревом.

Линтер, форматтер, сборщик — все сначала строят AST

Правило линтера — обход дерева

```
// правило: запретить console.log
export default {
  create(context) {
    return {
      CallExpression(node) {
        if (node.callee.object?.name === "console") {
          context.report({ node, message: "Уберите console" });
        }
      },
    };
  },
};
```

ESLint обходит AST: правило — посетитель узлов определённого типа

ESLint находит ошибки

```
function castSpell(mage, spell) {
  if (mage.mana = spell.cost) {
    mage.mana -= spel.cost;
  }
}

$ npx eslint src/bad.js
1:10 error 'castSpell' is defined but never used  no-unused-vars
2:7  error Expected a conditional expression and
           instead saw an assignment              no-cond-assign
3:18 error 'spel' is not defined                  no-undef
```

Линтер читает код, не запуская его: ошибку находит он, а не пользователь

Конфигурация ESLint

```
// eslint.config.js
import js from "@eslint/js";
import globals from "globals";
import { defineConfig, globalIgnores } from "eslint/config";

export default defineConfig([
  globalIgnores(["dist"]),
  js.configs.recommended,
  { languageOptions: { globals: globals.browser } },
]);
```

Конфиг — JavaScript-модуль: набор правил и окружение

Prettier: формат вместо споров

```
// до
const mage={name:"Маг",level:1,spells:["Искра","Щит"]}
function heal(m){return {...m,hp:m.maxHp}}

$ npx prettier . --write

// после
const mage = { name: "Маг", level: 1, spells: ["Искра", "Щит"] };
function heal(m) {
  return { ...m, hp: m.maxHp };
}
```

Prettier печатает AST заново в едином стиле

Три вида проверок

Инструмент	Что ищет	Пример
ESLint	подозрительный код и ошибки логики	присваивание в if
Prettier	только внешний вид	отступы, кавычки, переносы
TypeScript	несовпадение типов	строка вместо числа

ESLint — про смысл, Prettier — про вид, TypeScript — про типы

Мини-задача: проект с GitHub

```
$ git clone https://github.com/…/yellow-mage.git
$ cd yellow-mage
$ npm run dev
```

Что увидит человек, который только что склонировал проект?

Что нужно сделать?

Подумать 30 секунд, ответ на следующем слайде

Разбор: сначала зависимости

```
$ npm run dev
sh: 1: vite: not found

$ npm ci
$ npm run dev
  ➜  Local:   http://localhost:5173/
```

node_modules не хранится в git.

Зависимости восстанавливаются из lock-файла.

После clone — npm ci или npm install

Мини-задача: попадёт ли meteor

```
// spells.js
export function spark(power) { return power * 2; }
export function meteor() { return "METEOR_UNUSED"; }

// main.js
import * as spells from "./spells.js";
console.log(spells.spark(3));
```

Импортируем весь модуль целиком.

Останется ли meteor в сборке?

Подумать 30 секунд

Разбор: не попадёт

```
import * as spells from "./spells.js";
console.log(spells.spark(3));   // meteor выброшен

// а здесь сборщик не знает, что понадобится
console.log(Object.keys(spells));
doSomething(spells);
```

spells используется только как spells.spark — это видно статически.

Если объект модуля уходит наружу, оставить придётся всё.

Tree-shaking работает, пока использование видно из кода

Демонстрация после лекции

- npm create vite@latest yellow-mage -- --template vanilla
- Перенос кода «Жёлтого мага» в src/, конфетти через canvas-confetti
- Dev-сервер: вкладка Network, переписанные импорты, HMR для CSS
- ESLint и Prettier на проекте, npm run lint
- npm run build: содержимое dist, хеши, npm run preview

Готовый проект yellow-mage прилагается

Что важно унести с собой

- Браузер понимает только URL, имена пакетов разрешают Node и сборщики
- Статические import дают граф модулей и tree-shaking
- package.json — что подходит, lock-файл — что установлено
- Сборка: бандлинг, минификация, хеши, source maps, транспиляция
- Vite: в разработке модули по запросу, в production — Rolldown
- Линтер и форматтер работают с AST и ничего не запускают

Следующий шаг: React на проекте Vite

Источники и дальнейшее чтение

- MDN: JavaScript modules, import maps, tree shaking
- Node.js: ECMAScript modules, Packages, поле exports
- npm Docs: package.json, package-lock.json, npm ci, threats and mitigations
- Vite Guide: Why Vite, Dependency Pre-Bundling, HMR API, Build
- ESTree, AST Explorer; ESLint: Custom Rules
- Prettier: Rationale

Ссылки на конкретные разделы находятся в заметках слайдов